

National Tsing Hua University
Department of Electrical Engineering
EE6620 Computational Photography, Spring 2026

Homework #3

Super Resolution

Assigned on April 22, 2026

Due by May 13, 2026

Homework Description

Super-resolution is a class of techniques which can enhance the resolution of images. In this assignment, you will work on two types of super-resolution methods, **optimization-based** and **convnet-based**. For the optimization-based part, you will solve the SR problem with a single-image method and multi-image method. For the convnet-based part, you are going to build and train a SR model, then try to increase its size.



Low-resolution image



Optimization-based SR



Convnet-based SR

Table 1: Scores

Items		Score
Implementation (30%)	Single-Image SR	5%
	Multi-Image SR	10%
	Construct SR Model	10%
	Increase Model Size	5 %
Report (70%)	Exploration	20%
	Research study	30%
	Free Study	20%

For coding environment, we are using Google Colab as in the previous homework. You must submit through sharing the entire folder with vickychen910716@gmail.com as editor. Do not download and re-upload since coding histories would be checked. **Any work lacking editing history will be flagged for plagiarism or AI generation, leading to a 25% grade penalty.**

1 Implementation (30%)

In this section, you will work on the following four problems. Your output images should be saved in `/result`, and your trained models should be saved in `/model_trained`. Your results will be evaluated based on the PSNR metric. The high resolution images and TA's reference answers are provided in `/reference` for evaluation.

1.1 Single-Image Super Resolution (5%)

Super-resolution is an inverse problem and can be modeled as

$$Y = DBMX + n, \quad (1)$$

where Y is the observed low-resolution image, X is the latent high-resolution image, M is motion shift, B is blur, D is down-sampling and n is noise. We will use ProxImaL package [1] as in homework 2 to solve the latent image X . The objective function for single-image SR problem is defined as

$$E(X) = \|D(X \circledast B) - Y\| + \lambda \|\nabla X\|_{tv}, \quad (2)$$

$$\|\nabla X\|_{tv} = \sum_p \sqrt{X(p)_x^2 + X(p)_y^2}. \quad (3)$$

Specifically, Y is a provided low-resolution image, D is down-sampling by four, B is a Gaussian kernel and M is omitted since there is only one single image. The Gaussian kernel here is

$$B_{Gaussian}(C_x, C_y, k, l) = \frac{\exp(-\frac{\|(C_x, C_y) - (k, l)\|^2}{2\sigma_s^2})}{\sum_{k, l} B_{Gaussian}}, \quad k, l \in [-N/2, N/2] \quad (4)$$

where (C_x, C_y) is the kernel center index, and is equal to $(-1.5, -1.5)$ here, which means the kernel has a center shift away from $(0, 0)$.

The down-sampling by four process is that we sample one pixel from every four pixels along each dimension. Here we assume that each pixel in LR images is sampled from the center point of every 16 pixels in HR images. However, in actual implementation, *e.g.*, `subsample()` function provided in ProxImaL, it sample the first pixel (the upper left pixel) from every 16 pixels. The two different down-sample schemes are shown in Figure 1. Thus when convolving with the first pixel, the center point of kernel should align to the center point of the 16 pixels to match our assumption, which is illustrated in Figure 2. In the case of down-sampling by four, the kernel center shift would be $(+1.5, +1.5)$ pixels. The reason why the actual center shift is $(-1.5, -1.5)$ pixels in Eq. (4) is that when doing convolution, we should flip the kernel beforehand.

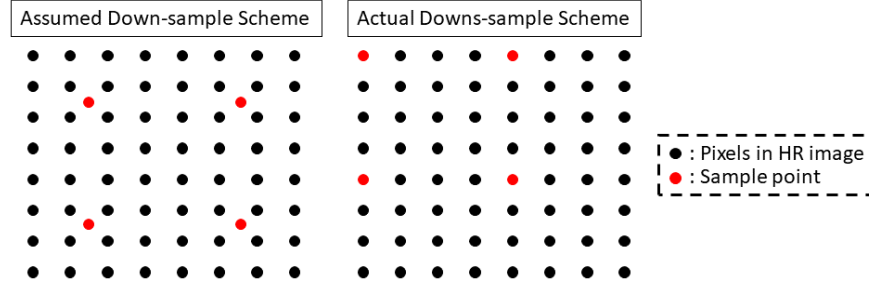


Figure 1: Different down-sample schemes

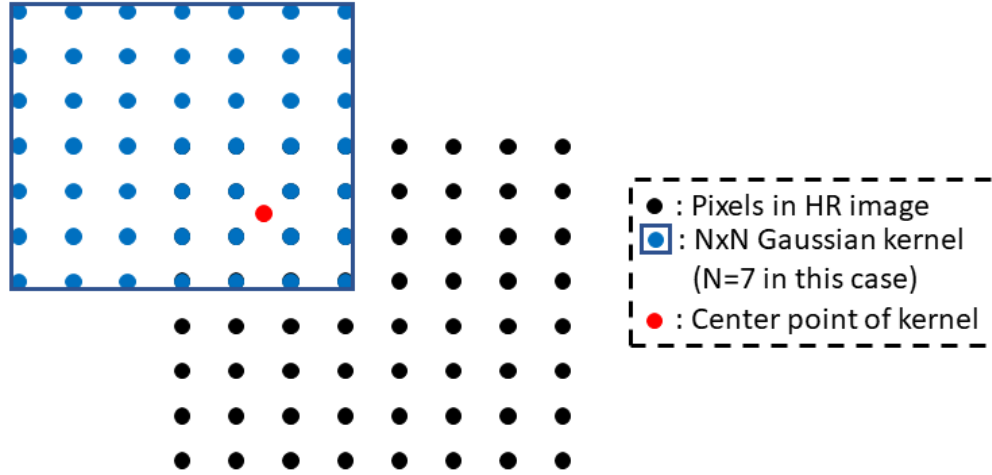


Figure 2: Alignment of kernel center when doing convolution

In this part, you will work in `/optimization-based` and need to complete the Gaussian kernel function `get_kernel()` in `sr_single.ipynb`. Your result for `/image/LR_zebra_test.png` should be similar to `/reference/zebra_test_single_golden.png` (PSNR>60dB), under the default setting $\{N, \sigma_s, \lambda\} = \{13, 1.2, 0.01\}$ to get a full score.

1.2 Multi-Image Super Resolution (10%)

Section 1.1 use one single LR image as the data. In this part, we provide **eight** LR images Y_i and they have motion shift relative to each other. The objective function for multi-image SR is defined as

$$E(X) = \left(\sum_i ||D(X \circledast B_i) - Y_i|| \right) + \lambda ||\nabla X||_{tv}, \quad i \in [0, 7] \quad (5)$$

$$||\nabla X||_{tv} = \sum_p \sqrt{X(p)_x^2 + X(p)_y^2}. \quad (6)$$

The effect of motion shift would be combined in the Gaussian kernel B_i , that is, you should further adjust the center point of each kernel based on the relative motion shifts. The kernel center due to both down-sampling and motion can be summarized as

$$C_x = -1.5 + 4 * mv_x, \quad (7)$$

$$C_y = -1.5 + 4 * mv_y, \quad (8)$$

where mv_x and mv_y are the relative motion shifts in LR images.

In this part, you will work in `/optimization-based` and need to complete the Gaussian kernel function `get_kernel()`, and formulate the optimization problem in `solve()` in `sr_eight.ipynb`. Your result for `/image/LR_zebra_test.png` should be similar to `/reference/zebra_test_eight_golden.png` (PSNR>60dB), under the default setting $\{N, \sigma_s, \lambda\} = \{15, 1.2, 0.01\}$ to get a full score.

1.3 Construct SR Model (10%)

In this part, we will work in `/convnet-based` and use PyTorch library [2] as a machine learning framework that helps us building and training convnet models. Here we construct the SR model with a popular building block - residual block [3], which has a Short-cut connection across one or more layers. Two example residual blocks are shown in Figure 3. C stands for base channel number (nFeat), and R stands for reduction ratio (ReduceRatio). **Note that we will set the closest integer as channel number if C/R is not an integer.** An advanced non-linear function - SiLU [4] is adopted for activation (see Figure 4). To learn the basic usage of PyTorch library, you can refer to the example code `ResBlock_typeA` in `model.ipynb` for Residual Block TypeA.

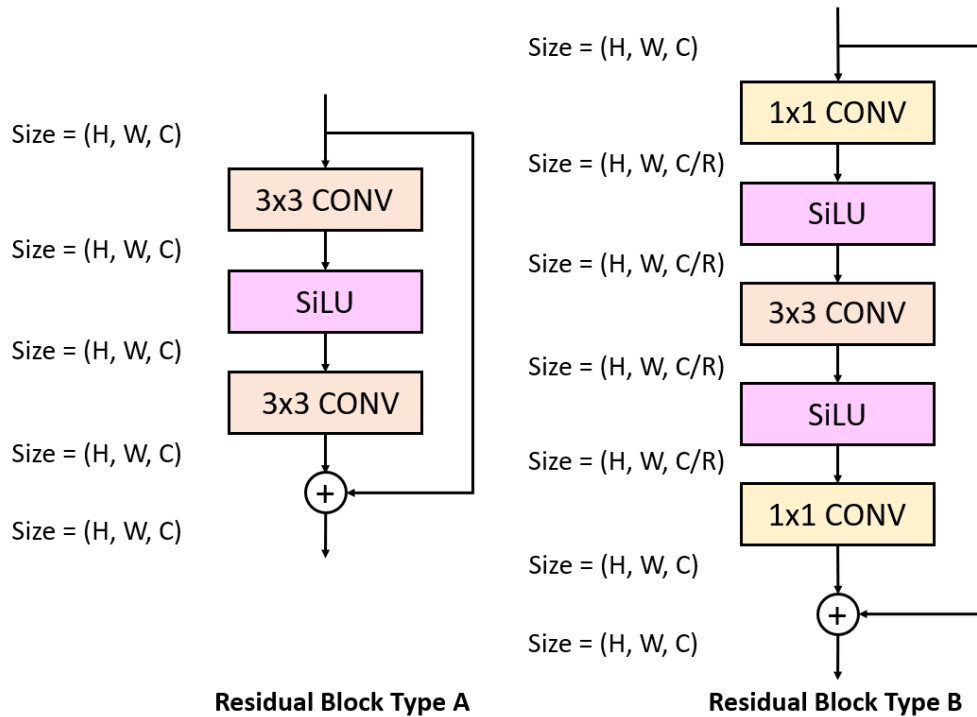


Figure 3: Example residual blocks

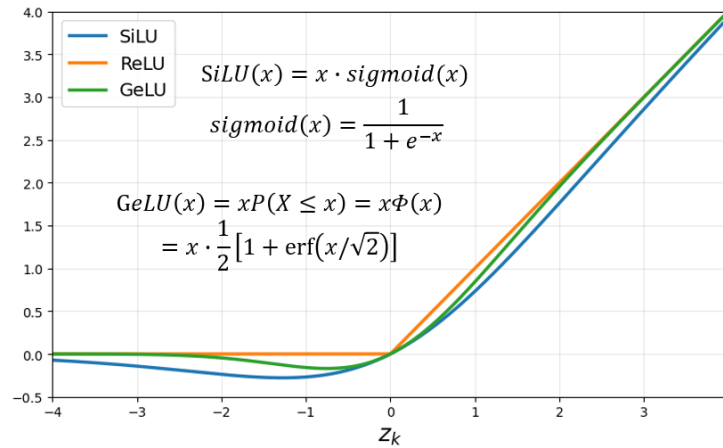


Figure 4: Formulation of SiLU and GeLU activation

The network architecture of our model called ZebraSRNet is shown in **Figure 5**. You need to construct this model by completing the class definition of `ResBlock_typeB`, `upsampler` and `ZebraSRNet` in `model.ipynb`. Note that we choose the **Residual Block Type B** for implementation of this assignment. After finishing constructing the model, you can train this model by running `train.ipynb`. Remember to set parameters as `{nFeat, ReduceRatio, nResBlock, nEpochs} = {32, 1, 3, 15}`.

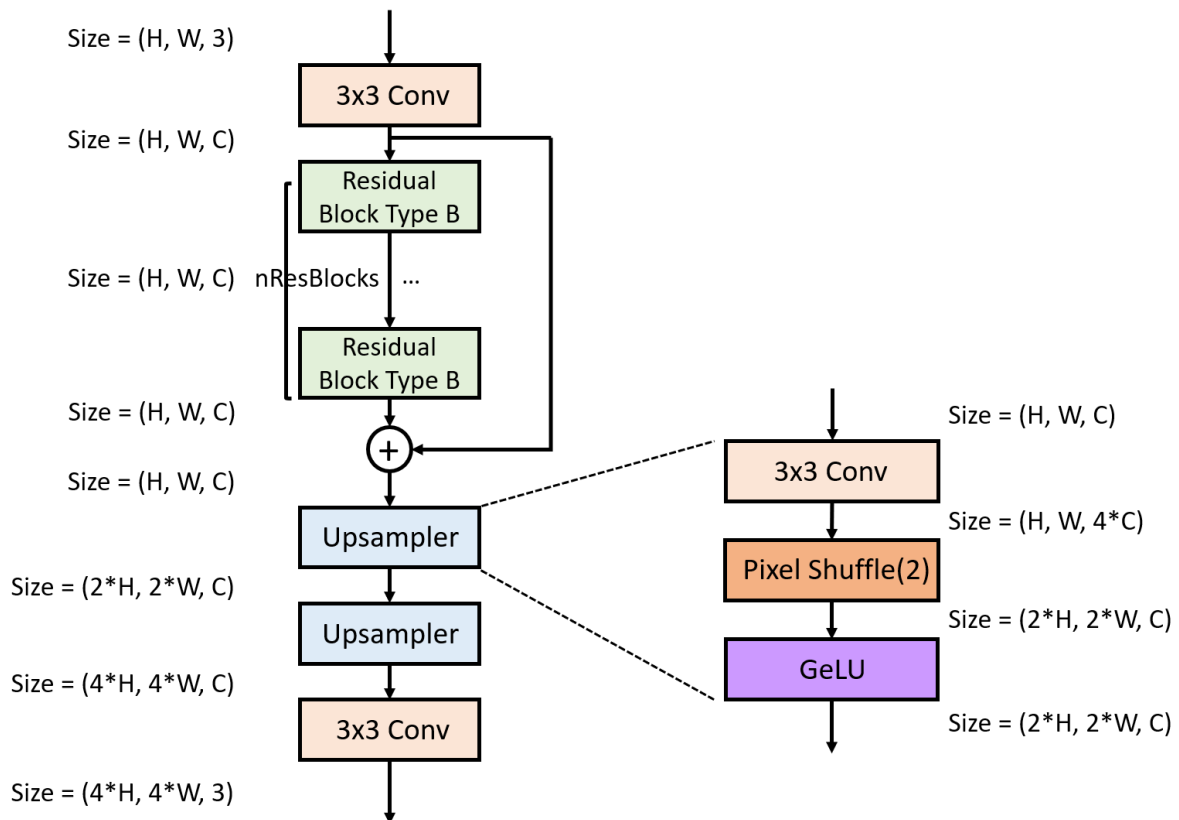


Figure 5: Network architecture of ZebraSRNet

Test your result by running `test.ipynb`. Your result for `/image_test/LR_zebra_test.png` should be similar to `/reference/F32B3R1_zebra_test_golden.png` (PSNR>40dB), under the setting `{nFeat, ReduceRatio, nResBlock, nEpochs}={32, 1, 3, 15}` to get a full score.

1.4 Increase Model Size (5%)

Based on the model you constructed in 1.3, we can increase the model size by increasing its width and depth. In this part, you need to modify the training parameters. Use `train.ipynb` to train the model and `test.ipynb` to evaluate it. Your result for `/image_test/LR_zebra_test.png` should be similar to `/reference/F64B8R2_zebra_test_golden.png` (PSNR>40dB), under the setting `{nFeat, ReduceRatio, nResBlock, nEpochs}={64, 2, 8, 120}` to get a full score.

2 Report (70%)

There are three parts of problems: Exploration, Research study, and Free study. Finish problems in Exploration and Research, then choose at least two interesting topics of super-resolution for your free study. **When conducting other experiments, please make sure not to overwrite the model checkpoints trained during the implementation part in `/model_trained`.**

2.1 Exploration (20%)

- a. (5%) In 1.3 and 1.4, TA has optimized the dataset loading behavior to accelerate the training process. Please examine the difference between `datasetTrain()` and `datasetTrain_speedup()` in `dataset.ipynb`, and explain why the optimized version achieves a faster runtime? You may replace `datasetTrain()` with `datasetTrain_speedup()` in `train.ipynb` and compare the runtime on both CPU and GPU over a few epochs (e.g., 3 epochs).
- b. (15%) Prepare your own training set and testing set by taking your own photos, and then train a model for super-resolution. How do you choose the training set and training flow? How your designed flow achieve your super-resolution task? **Remember that testing set should be different from training set. Using pre-exist datasets from the internet is prohibited. Your student ID card must be presented in the images or no points would be given.**

2.2 Research study (30%)

In this section, you should answer the problem by the two-step research flow: **Assumption** and **Justification**. Based on your observation, propose an assumption, and justify your idea by experiments. This research flow may be a loop:

observation→ propose assumption→ justification→modify assumption→justification ...

After reaching a satisfactory result, you need to describe your assumption and how you justify it by experiments in the report. Grading criteria for this part are based on clarity of the report, rigorousness of the justification and the overall research flow.

- a. (10%) In 1.2, we use eight images to solve the SR problem. What if we use fewer images? Is that using more images results in better quality? If not, try to propose assumptions and justify through experiments. You should select two, four, and six images from these eight images and conduct the experiments.
- b. (10%) Compare subjective and objective quality for different SR method, *i.e.*, direct interpolation, optimization-based and convnet-based. What's the pros and cons of different methods? What's the applicable scene of these methods? Try to justify your conclusion with further examples instead of only using test images in this assignment.
- c. (10%) What's the difference between two quality metrics, SSIM and PSNR? Are these metrics always consistent with human's intuitions? Try to justify your conclusion with further examples instead of only using test images in this assignment.

2.3 Free study (20%)

In this section, please propose at least **two** interesting aspects of super-resolution as your research topics, and answer it by the two-step research flow. Though it's encouraged to propose on your own, you can still choose your free study topic from the following problems. **TA will consider both the breadth and depth of the free study, choosing the top two students. And 2% and 1% bonus scores on semester will be given to the best and the second-best students.**

- a. Test the model you trained in both 1.3 and 1.4 with the provided images in [image_hidden/](#). What happen to the results? Why the results look weird? Try to resolve the artifacts and provide justification for how or how not your method work.
- b. Try different parameter settings $\{N, \sigma_s, \lambda\}$ in 1.2. What's the effect of each parameter?
- c. Why the kernel center shifts for multi-image SR can be expressed as Eq. (7) and Eq. (8)?
- d. Taking your own photos as the data and perform multi-image SR as in 1.2. How do you take those photos? How do you estimate the motion shift of each photo?
- e. Design and implement a convnet-based framework for multi-image SR and compare it with the optimization-based approach and single-image SR. It would be best to highlight how you address the misalignment between images.

3 Deliverable

1. Submit only your report with filename `{studentID}_report.pdf` to eclass.

2. Modify your folder name to `HW3_{studentID}`, and share with vickychen910716@gmail.com as editor. (Folder structure shall be identical to the assigned homework.)
 - (a) Your `sr_single.ipynb`, `sr_eight.ipynb`, `model.ipynb` and any `.ipynb` written for the report.
 - (b) Your result super-resolution images in folder `/result`.
 - (c) Your trained model in folder `/model_trained` and the training logs in `/convnet-based`.
 - (d) Images you've taken for training and testing in folder `/own image`.
 - (e) Your report in Google Doc.

Note that editing histories will be reviewed. As mentioned earlier, if no history is found, the work may be considered generated by AI or plagiarized, resulting in a 25% penalty. Additionally, wrong file delivery or arrangement will get 5% punishment.

References

- [1] F. Heide, S. Diamond, M. Nießner, J. Ragan-Kelley, W. Heidrich, and G. Wetzstein, "Proximal: Efficient image optimization using proximal algorithms," *ACM Trans. Graph.*, vol. 35, July 2016.
- [2] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32* (H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, eds.), pp. 8024–8035, Curran Associates, Inc., 2019.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- [4] S. Elfving, E. Uchibe, and K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *CoRR*, vol. abs/1702.03118, 2017.